

Lezione 22 parte 2 - Spring framework

Da HTTP a Spring

Abbiamo visto come funziona il [protocollo HTTP](#), come sono strutturate le request e le response, cosa sono i verbi e gli status code.

Abbiamo anche capito che le applicazioni a servizi REST si basano su questo protocollo per scambiare dati in formato JSON.

Ma come si costruisce concretamente un server che riceva queste richieste, le elabori e restituisca le risposte corrette?

Farlo da zero in Java puro sarebbe estremamente complesso: bisognerebbe gestire manualmente i socket di rete, il parsing delle richieste HTTP, la serializzazione degli oggetti Java in JSON e molto altro.

È esattamente qui che entra in gioco Spring:

- un framework che si occupa di tutta questa infrastruttura al posto nostro, permettendoci di concentrarci esclusivamente sulla logica applicativa.

Spring Framework

Cos'è un Framework?

Prima di parlare di Spring è importante capire cosa distingue un **framework** da una semplice **libreria** — una distinzione che hai già incontrato indirettamente con `i Thread` e il `toString()`.

Una **libreria** è un componente passivo:

- sei tu a istanziarla, popolarla e richiamare i suoi metodi nel momento che decidi.
- `ArrayList`, `PreparedStatement`, `DriverManager` — li abbiamo sempre usati noi, chiamandoli esplicitamente nel codice.

Un **framework** funziona al contrario:

- implementa il principio di **inversione di controllo** ([IoC — Inversion of Control](#)).
- Ovvero Non sei tu a chiamare il framework — è il framework a chiamare il tuo codice nel momento giusto.

Senza saperlo in realtà abbiamo già visto questo meccanismo:

- Il metodo `toString()` :
 - **non lo si chiama mai esplicitamente — viene chiamato automaticamente da Java quando stampi un oggetto**
- Il metodo `start()` dei Thread:
 - **non esegue direttamente il tuo codice — è la JVM a decidere quando eseguire `run()`**

Con Spring il principio è lo stesso ma applicato su scala più ampia:

- **non esiste più un `main` che istanzia e coordina tutto.**
- **Sarai tu a definire le classi e i loro comportamenti, ma sarà Spring a istanziarle, collegarle tra loro e invocare i metodi nel momento opportuno.**

I Progetti di Spring

Spring non è un singolo strumento ma una **famiglia di progetti**, ognuno dedicato a una problematica specifica dello sviluppo enterprise:

- **Spring Web — per applicazioni web a pagine o a servizi REST**
- **Spring Data — per la gestione della persistenza (sostituisce JDBC)**
- **Spring Mobile — estensione di Spring Web per applicazioni mobile**
- **Spring Integration — per l'integrazione di applicazioni aziendali**
- **Spring Security — per la gestione di autenticazione e autorizzazione**

Ognuno di questi progetti implementa l'inversione di controllo nel suo ambito specifico.



Spring Boot e Maven

Spring Boot è il punto di partenza di ogni progetto Spring moderno.

Il suo compito principale è gestire automaticamente le dipendenze nel `pom.xml`

— si occupa di scaricare e configurare tutti i `jar` necessari per far funzionare Spring, sollevandoti dalla responsabilità di farlo manualmente.

Un progetto **Maven** con Spring Boot è quindi:

- un progetto Maven che dipende da Spring Boot, il quale a sua volta gestisce tutte le dipendenze dei vari moduli Spring di cui hai bisogno.

Spring Web e il Pattern MVC

Spring Web è:

- il modulo di Spring dedicato alla realizzazione di applicazioni web. Si basa sul pattern architetturale **MVC** (Model-View-Controller), che propone di scomporre il sistema in tre livelli con responsabilità ben separate.

I Tre Livelli dell'MVC

1. Model — La Logica di Business (Backend) È il cuore dell'applicazione:

- contiene le regole di business, gestisce i dati e interagisce con il Data Layer.

🔗 **Corrisponde a tutto ciò che abbiamo visto finora:**

il 'LibroDAO', il 'LibroDTO', la connessione al database.

2. View — Lo Strato di Presentazione (Frontend):

- È l'interfaccia con cui l'utente interagisce — pagine HTML, app mobile, o qualsiasi altro client.
- Non contiene logica applicativa: **si limita a presentare i dati che riceve e a raccogliere gli input dell'utente.**

3. Controller — Il Coordinatore:

- È il componente più importante dal punto di vista architetturale:
 - **fa da centralino e mediatore tra Model e View.**
 - **Il principio fondamentale è che Model e View non si parlano mai direttamente — qualsiasi comunicazione tra i due passa sempre attraverso il Controller.**

Il Controller ha quindi due responsabilità:

1. **È il punto di ingresso per le richieste dei client**
2. **Disaccoppia Model e View, garantendo che ognuno dei due non sappia nulla dell'altro**

Il Flusso di Comunicazione



Plain text

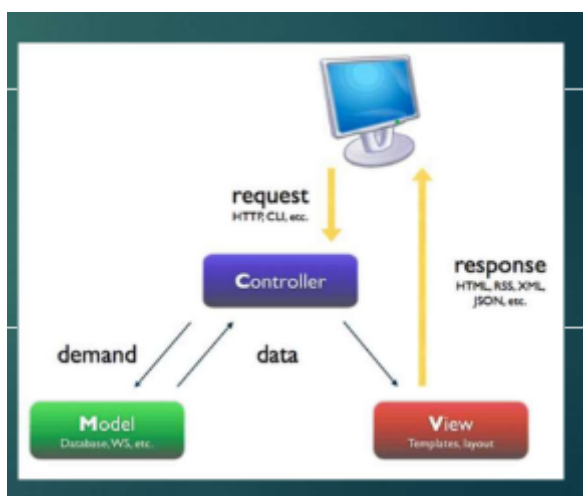
1 View → Controller → Model → Controller → View

1. **La View invia una richiesta al Controller**
2. **Il Controller la elabora e la inoltra al Model**
3. **Il Model esegue la logica di business e restituisce il risultato al Controller**
4. **Il Controller passa il risultato alla View per la presentazione**

🔗 **Nota - il collegamento con l'architettura three-tier vista in precedenza:**

il Controller corrisponde al Logic Layer, il Model interagisce con il Data Layer, e la View è il Client Layer.

MVC è quindi la realizzazione concreta di quell'architettura a livello di codice.



Guardando l'architettura completa, il flusso di una richiesta si articola in questi passaggi:

1. **Il Client invia una richiesta HTTP al Controller**

2. **Il Controller non elabora nulla da solo — delega al Model la richiesta di dati o servizi**
3. **Il Model esegue la logica di business, accede ai dati e restituisce il risultato al Controller**
4. **Il Controller passa il risultato alla View**
5. **La View costruisce la risposta finale — che sia HTML, JSON o altro — e la invia al Client**

Il punto fondamentale è che **nessuno dei tre componenti parla direttamente con gli altri due** senza passare dal Controller.

Il Model non sa nulla della View, la View non sa nulla del Model, e il Client non interagisce mai direttamente né con il Model né con la View.

Il Controller è l'unico punto di contatto tra tutti — fa da **dogana**:

- **filtra, smista ed elabora sia le richieste in entrata dal Client che le risorse in uscita dal Model, prima di passarle alla View.**

Questo disaccoppiamento è il vero valore del pattern MVC:

- **se domani si vuole cambiare la View da HTML a JSON, o sostituire il database nel Model, il Controller rimane invariato — e viceversa.**

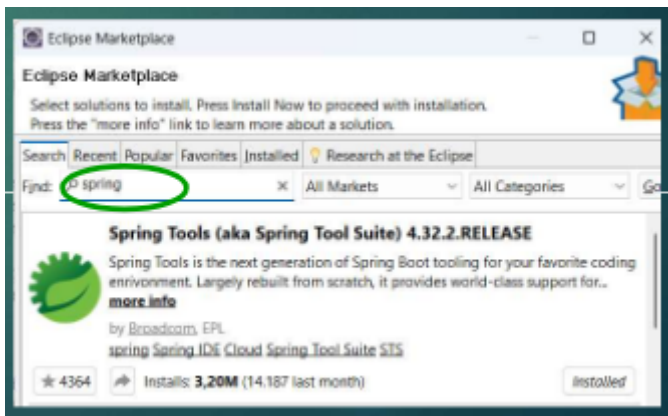
Spring Boot

Per utilizzare Spring Web è necessario scaricare le librerie relative al modulo. Esistono tre modi per farlo, in ordine crescente di praticità:

1. **Scaricarle a mano — scelta sconsigliata:**
 - **oltre alle librerie di Spring bisogna scaricare manualmente tutte le sotto-dipendenze, un processo lungo e soggetto a errori.**
2. **Creare un progetto Maven — scelta consigliata:**
 - **Maven gestisce automaticamente tutte le sotto-dipendenze dichiarate nel `pom.xml`.**
3. **Creare un progetto Spring Boot — scelta ottimale:**
 - **configura Maven automaticamente e lo attiva, occupandosi di tutta la configurazione iniziale del progetto.**

Creare un Progetto Spring Boot su Eclipse

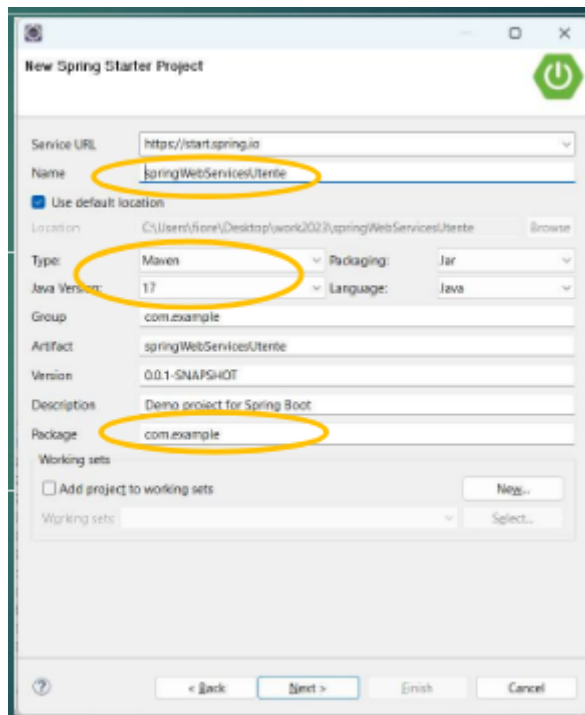
1. **Installare il plugin Spring Tools:** Menu Help → Eclipse Marketplace → Cercare "Spring Tools 5.0.1 Release" → Installa



2. Creare il progetto: Menu File → Project → Cartella Spring Boot → Spring Starter Project

3. Configurare il progetto:

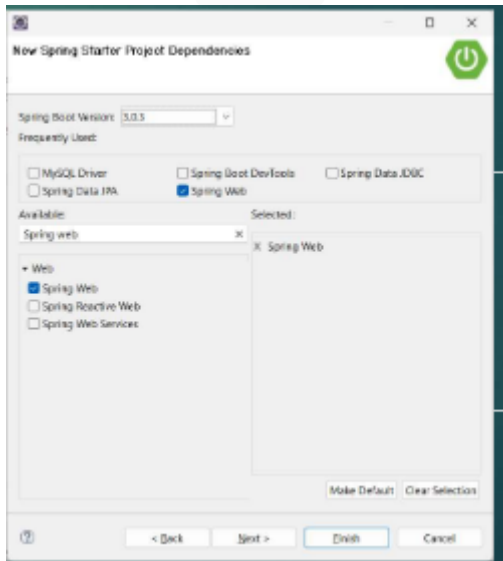
- **Name** — nome del progetto
- **Type** — selezionare Maven (gestione delle dipendenze)
- **Java Version** — dalla versione 17 in su
- **Package** — seguire la convenzione `com.nomeSocietà.nomePacchetto`



4. Aggiungere le dipendenze:

- Cliccare su Next
- Nella casella di ricerca cercare Spring Web
- Flaggare la casella corrispondente

- Cliccare su `Finish`



Il plugin Spring Tools 4.x richiede una versione di Eclipse **>= 2021**.

Struttura del Progetto Spring Boot

Una volta creato il progetto, Eclipse genera automaticamente questa struttura:



CSS

```

1  springWebServicesUtente/
2  |   src/main/java/
3  |       SpringWebServicesUtenteApplication.java  # classe starter
4  |   src/main/resources/
5  |       application.properties                  # file di configurazione
6  |   pom.xml                                     # gestione delle dipendenze

```

Nota importante:

La directory `src/main/webapp`, tipica delle web application tradizionali, **non è presente** — perché non produciamo pagine HTML ma dati in formato JSON.

Configurare il `pom.xml`

Creando il progetto con Spring Boot, vengono caricate automaticamente le dipendenze base:

- `spring-boot-starter-web` — **il modulo Spring Web**
- `spring-boot-starter-test` — **per i test**

Di default il progetto produce dati in formato **JSON**. Se si vuole supportare anche **XML** bisogna aggiungere manualmente questa dipendenza:



XML

```
1 <dependency>
2   <groupId>com.fasterxml.jackson.dataformat</groupId>
3   <artifactId>jackson-dataformat-xml</artifactId>
4 </dependency>
```

Configurare application.properties

Spring ha una **Servlet Controller** interna che fa da punto di ingresso per tutte le richieste HTTP — non è visibile nel codice del progetto, ma è configurabile tramite `application.properties`:



.properties

```
1 # configuro l'url della servlet controller di Spring
2 spring.mvc.servlet.path=/spring-utenti
```

Se non impostata, l'URL di default sarà `/`. Per iniziare non servono altre configurazioni.

La Classe Controller

Le classi Controller sono **semplici classi Java**:

- **non estendono né implementano nulla di specifico.**
Vengono riconosciute e gestite da Spring tramite **annotazioni**:



Java

```
1 @RestController
2 @RequestMapping(path = "/gestioneutenti")
3 public class UtenteController {
4     // metodi della controller
5 }
```

- `@RestController` :
 - **segnala a Spring che questa classe è un Controller.**
 - **Spring la istanzierà automaticamente allo startup del server, senza che tu debba mai chiamare** `new UtenteController()` :

- è l'**Inversione di controllo** in azione.
- `@RequestMapping(path = "")` :
 - **definisce il path base del Controller.**
 - **Tutte le richieste che iniziano con `/gestioneutenti` verranno gestite da questa classe.**

Nota:

la classe non ha dipendenze esplicite da Spring se non per le annotazioni — è codice Java puro, reso "speciale" solo dalle annotation.

I Metodi della Controller

Ogni metodo della Controller gestisce un **verbo HTTP specifico**, dichiarato tramite annotazione:

Annotazione	Verbo HTTP
<code>@GetMapping</code>	GET — lettura
<code>@PostMapping</code>	POST — inserimento
<code>@PutMapping</code>	PUT — modifica totale
<code>@PatchMapping</code>	PATCH — modifica parziale
<code>@DeleteMapping</code>	DELETE — cancellazione

Ogni mapping specifica anche:

- `path` — **l'URL del metodo, che si aggiunge al path del Controller**
- `produces` — **il formato dei dati restituiti**
- `consumes` — **il formato dei dati attesi in input**

L'URL completa che identifica univocamente un metodo è:



Plain text

```
1 path del Controller + path del metodo
```

Esempio:



Java

```
1 // GET: lettura di un utente per id
2 // L'id viene passato direttamente nell'URL come path variable
3 @GetMapping(path = "/cerca/{idUtente}", produces =
  "application/json")
4 public Utente cercaUtente(@PathVariable Integer idUtente) {
5     // funzione di ricerca
6 }
7
8 // POST: inserimento di un nuovo utente
9 // I dati dell'utente vengono passati nel body della richiesta in
  formato JSON
10 @PostMapping(path = "/registra", consumes = "application/json")
11 public void registra(@RequestBody Utente utente) {
12     // funzione di inserimento
13 }
```

Tre concetti chiave da notare:

`{idUtente}` e `@PathVariable` :

- la variabile nell'URL tra parentesi graffe corrisponde all'argomento del metodo annotato con `@PathVariable` .
- Spring estrae automaticamente il valore dall'URL e lo passa al metodo.

`produces` :

- indica che il metodo restituisce un oggetto `Utente` che Spring serializzerà automaticamente in JSON.

`consumes` ed `@RequestBody` :

- indica che il metodo si aspetta un oggetto JSON nel body della richiesta.
- Spring lo deserializzerà automaticamente nell'oggetto `Utente` annotato con `@RequestBody` .

Primo Esempio — CalcolatriceController

Questo è il primo esempio pratico di un Controller Spring.

È volutamente semplice — espone solo operazioni matematiche tramite metodi

GET — ma permette di vedere concretamente tutti i concetti che abbiamo appena studiato.



Java

```
1  @RestController
2  @RequestMapping(path="/home")
3  public class CalcolatriceController {
4
5      public CalcolatriceController() {
6          System.out.println("Sta costruendo
calcolatriceController");
7      }
8
9      @GetMapping(path="/somma")
10     public int somma(int a, int b) {
11         return a + b;
12     }
13
14     @GetMapping(path="/sottrazione")
15     public int sottrazione(int n, int m) {
16         return n - m;
17     }
18
19     @GetMapping(path="/molti")
20     public int moltiplicazione(int n, int m) {
21         return n * m;
22     }
23
24     @GetMapping(path="/divisione")
25     public double divisione(int a, int b) {
26         if(b <= 0) {
27             throw new ArithmeticException("Errore! Divisione per
zero");
28         }
29         return a / b;
30     }
31 }
```

Analisi

Il costruttore

Il costruttore è esplicito ma non fa nulla di speciale — serve solo a dimostrare

l'inversione di controllo in azione.

Non siamo noi a chiamare `new CalcolatriceController()` :

- **è Spring a istanziare la classe automaticamente allo startup del server, invocando il costruttore a zero argomenti.**
- Il `System.out.println` lo conferma:
 - **quando avvii il server, vedrai il messaggio in console prima ancora di fare qualsiasi richiesta.**

Le annotazioni

- `@RestController` — **Spring riconosce questa classe come Controller e la gestisce**
- `@RequestMapping(path="/home")` — **tutte le richieste che iniziano con /home verranno gestite da questa classe**

Il passaggio dei parametri

A differenza degli esempi precedenti con `@PathVariable` e `@RequestBody`, qui i parametri non hanno nessuna annotazione.

Questo significa che Spring si aspetta di riceverli come query parameters — accodati all'URL dopo `?` :



HTTP

```
1 GET http://localhost:8080/home/somma?a=5&b=3
2 GET http://localhost:8080/home/sottrazione?n=10&m=4
3 GET http://localhost:8080/home/moltiplicazione?n=3&m=6
4 GET http://localhost:8080/home/divisione?a=10&b=2
```

Spring mappa automaticamente i parametri dell'URL sugli argomenti del metodo **per nome**:

- **a nell'URL corrisponde al parametro a del metodo, b a b, e così via.**

La gestione degli errori:

Il metodo `divisione()` introduce un primo esempio di gestione degli errori:

- **se b è minore o uguale a zero viene lanciata una `ArithmeticException`.**
In una API REST questo non è però il modo corretto di gestire gli errori:
- **vedremo in seguito come restituire al client uno status code appropriato (ad esempio `400 Bad Request`) invece di lasciare che sia Spring a**

gestire l'eccezione in modo generico.

Scomporre il Model — L'Architettura Completa

Il pattern MVC afferma che il **Controller** deve fare solo coordinamento — **non deve contenere logica di business**.

L'esempio della calcolatrice, per quanto didatticamente utile, violava già questo principio: i metodi `somma()`, `divisione()` ecc. contenevano logica che in un'applicazione reale non dovrebbe stare nel Controller.

Il Controller deve essere un **centralino**:

- **riceve la richiesta, la smista al componente giusto, e restituisce la risposta.**

Niente di più.

Per questo il **Model** viene tipicamente scomposto in tre layer con responsabilità distinte:

1. Layer Entity:

- Classi java che modellano le **entità del dominio**:
 - **corrispondono direttamente alle tabelle del database.**
 - È esattamente il ruolo che abbiamo già visto svolgere al 'LibroDTO' nell'esempio JDBC:
 - **una classe con i campi della tabella e i relativi getter e setter, senza logica.**

2. Layer DAO:

- Classi che contengono la **logica di basso livello per l'accesso ai dati persistenti.**
- **Dialogano direttamente con il database tramite JDBC o con un ORM come Hibernate.**
- Anche questo ruolo lo conosciamo già — **è esattamente il 'LibroDAO' che abbiamo scritto in precedenza**.

3. Layer Service:

- **Classi che contengono la logica di business ad alto livello.**
- Il Service è il nuovo strato che si inserisce tra il Controller e il DAO:

- riceve le richieste dal Controller, applica le regole di business, e delega al DAO le operazioni sui dati.

Nota:

tra Service e DAO vengono scambiate le Entity.

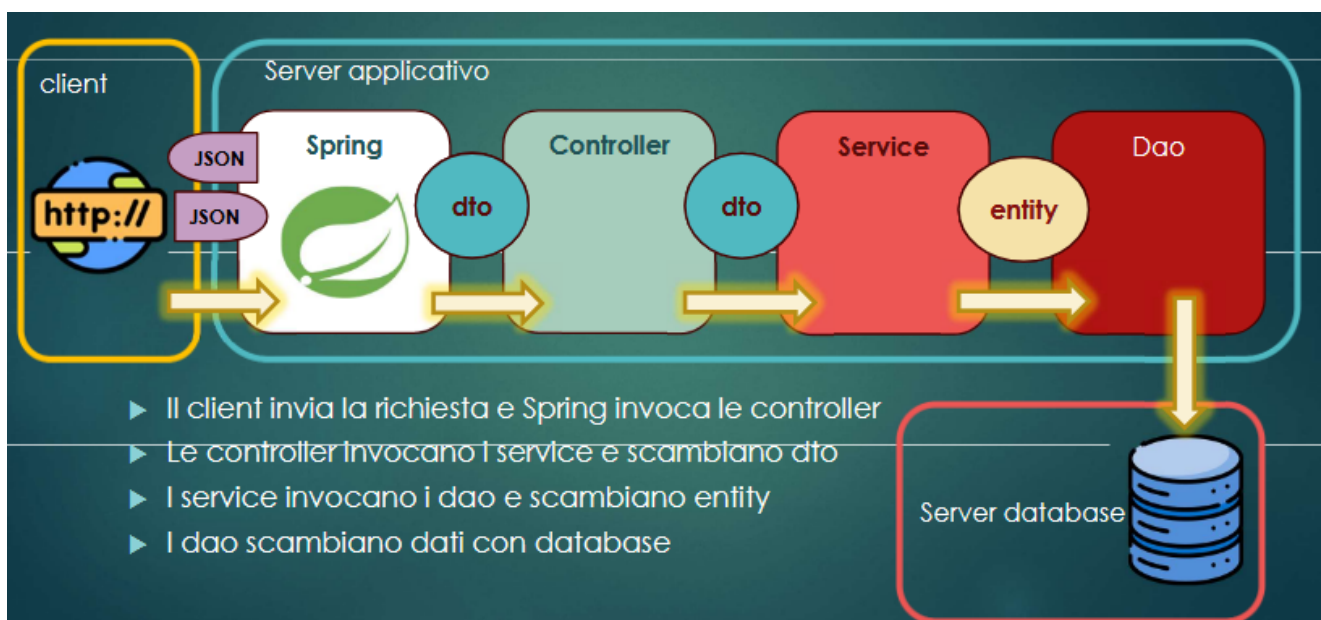
Come il DAO accede concretamente al database — tramite JDBC, Hibernate o altro — non interessa al Service:

- è un dettaglio implementativo nascosto dietro l'interfaccia del DAO.

Ritroviamo ancora una volta il principio di **Single Responsibility** e il **pattern Adapter** che abbiamo visto fin dall'inizio con JDBC.

L'architettura

Ora che abbiamo spiegato in generale questa architettura , possiamo finalmente mettere tutto insieme e seguire il flusso completo di una richiesta dall'inizio alla fine.



1. Il Client invia la richiesta a Spring:

- Il client invia una richiesta HTTP con un payload in formato JSON.
- Spring — tramite la sua Servlet Controller interna — riceve la richiesta, deserializza il JSON in un oggetto DTO e lo passa al Controller corretto in base all'URL e al verbo HTTP.

2. Spring invoca il **Controller**:

- Il Controller riceve il **DTO** dal layer Spring e lo usa per capire cosa fare.
- Come abbiamo detto, il **Controller** è solo un centralino — non contiene logica di business:

- Si limita a delegare l'operazione al Service corretto, passandogli il **DTO**.

3. Il **Controller** invoca il **Service** scambiando **DTO**:

- Tra Controller e Service viaggiano i **DTO**:
 - gli oggetti che rappresentano i dati così come li vede il client.
 - Il Service riceve il DTO, applica la logica di business, e quando ha bisogno di accedere ai dati delega al DAO.

4. Il Service invoca il DAO scambiando Entity:

- Qui avviene un passaggio importante:
 - tra Service e DAO non viaggiano più i DTO ma le Entity — la rappresentazione interna dei dati che corrisponde alle tabelle del database.
- È il Service a fare da traduttore tra i due mondi:
 - converte il DTO in Entity prima di passarlo al DAO, e converte l'Entity in DTO prima di restituirlo al Controller.

5. Il DAO dialoga con il Database:

- Il DAO è l'unico componente che parla direttamente con il database — tramite JDBC o un ORM come Hibernate.
- Esegue le query, recupera i dati grezzi e li restituisce al Service sotto forma di Entity.

Perché questa separazione?

Ogni freccia nell'immagine rappresenta un confine preciso con un tipo di oggetto ben definito:

- Spring ↔ **Controller** — viaggiano **DTO** (JSON deserializzato)
- **Controller** ↔ **Service** — viaggiano **DTO**
- **Service** ↔ **DAO** — viaggiano Entity
- **DAO** ↔ Database — viaggiano dati grezzi (righe SQL)

Questo significa che ogni layer conosce solo i layer adiacenti e parla solo il loro linguaggio.

- Il Controller non sa nulla del database, il DAO non sa nulla del client.
 - Se domani si cambia il database, si tocca solo il DAO.
 - Se cambia il formato dei dati verso il client, si tocca solo il DTO.
- Il resto rimane invariato.

Ritroviamo ancora una volta il principio di **Single Responsibility** e il **pattern Adapter** che abbiamo visto fin dall'inizio con **JDBC** — solo ora applicati su scala architeturale completa.

II DTO — Data Transfer Object

Le classi Controller scambiano dati con il client tramite oggetti Java che Spring trasforma automaticamente da Java a JSON e viceversa.

Le classi usate per questo scopo sono i **DTO (Data Transfer Object)**.

DTO vs Entity

I DTO sono simili alle classi Entity ma con una differenza fondamentale:

- **mentre le Entity corrispondono direttamente alle tabelle del database (ogni Entity è una tabella, ogni oggetto è una riga),**
- **i DTO rappresentano i dati così come vengono scambiati con il client — che non necessariamente corrispondono a una singola tabella.**

Si può pensare ai DTO come alle possibili query o incroci tra tabelle:

- **una query su una singola tabella, una join tra più tabelle, un sottoinsieme di colonne.**

Ad esempio un DTO `StudenteConEsami` **potrebbe raggruppare i dati dell'entity `Studente` insieme agli esami superati — un'informazione che non esiste in una singola tabella.**

Perché non usare direttamente gli Entity?

Ci sono tre motivi precisi per cui gli Entity non devono mai uscire dallo strato Model:

1. Gli Entity sono mappati sulle tabelle:

- Se si vuole restituire al client il risultato di una join tra più tabelle, non esiste un Entity che lo rappresenti — **serve un DTO apposito che raggruppi i dati di più Entity.**

2. Gli Entity sono disaccoppiati dallo strato Controller:

- **Il Controller non deve vedere gli Entity:**
 - è un dettaglio implementativo del Model.
 - **Passare un Entity al Controller significherebbe rompere il disaccoppiamento tra i layer che abbiamo costruito con tanta cura.**

3. Gli Entity potrebbero essere collegati a un ORM:

- In sistemi che usano **Hibernate** — l'ORM più diffuso con Spring — gli Entity hanno un ruolo speciale:
 - **quando si modifica un Entity, Hibernate aggiorna automaticamente il database senza bisogno di scrivere il DAO manualmente.**
 - Per questo motivo gli Entity non devono essere "toccati" fuori dal Model — **modificarli nel Controller o passarli al client potrebbe causare comportamenti inaspettati sul database.**

☰ In sintesi:

- **gli Entity sono la rappresentazione interna dei dati nel Model,**
 - **i DTO sono la rappresentazione esterna dei dati verso il client.**
- Sono due cose diverse con due responsabilità diverse — **e tenerle separate è esattamente il principio di Single Responsibility che abbiamo visto dall'inizio.**

Il Service

Il **Service** è il layer centrale dell'architettura:

- **è il componente che contiene la logica di business ad alto livello.**
- **Si posiziona tra il Controller e il DAO**

Ha due responsabilità principali:

1. Contenere la logica di business:

- **Tutto ciò che non è né coordinamento (Controller) né accesso ai dati (DAO) vive nel Service.**

☰ Ad esempio:

validare i dati di un utente prima di inserirlo, calcolare il prezzo finale di un ordine applicando sconti, verificare che uno studente abbia i requisiti per sostenere un esame.

Queste sono regole di business — appartengono al Service.

2. Fare da traduttore tra Controller e DAO:

Come abbiamo già visto nell'architettura completa, **il Service è l'unico componente che conosce sia i DTO che le Entity.**

Riceve i DTO dal Controller, li converte in Entity per il DAO, e fa il percorso inverso quando restituisce i dati.

Questo lo rende il **fulcro** dell'intera architettura.

In Spring il Service è una semplice classe Java annotata con `@Service`, **che segnala a Spring di istanziarla e gestirla tramite inversione di controllo — esattamente come `@RestController` per il Controller.**



Java

```
1  @Service
2  public class UtenteService {
3
4      private UtenteDAO utenteDAO;
5
6      // il Service usa il DAO per accedere ai dati
7      public UtenteDTO cercaUtente(int id) {
8          // chiama il DAO che restituisce un Entity
9          Utente entity = utenteDAO.findById(id);
10         // converte l'Entity in DTO e lo restituisce al
            Controller
11         return UtenteMapper.daEntityADTO(entity);
12     }
13 }
```

Nota - il parallelismo con tutto ciò che abbiamo visto finora:

il Service è un'altra applicazione del principio di **Single Responsibility** — ha una sola ragione per cambiare, ovvero la logica di business .
Se cambia il database si tocca il DAO, se cambia il formato dei dati verso il client si tocca il DTO, se cambiano le regole di business si tocca il Service.

Il DAO nel contesto Spring

Come abbiamo già visto nell'esempio con JDBC, il **DAO** (Data Access Object) è:

- il componente che contiene la logica di basso livello per l'accesso ai dati persistenti — è l'unico layer che parla direttamente con il database.

In un'applicazione Spring il DAO rimane concettualmente identico a quello che abbiamo scritto con JDBC, ma con una differenza importante:

- in sistemi che usano Hibernate, il DAO non va scritto manualmente.
- Hibernate lo genera automaticamente a partire dalle classi Entity — basta definire le Entity e Hibernate si occupa di creare tutte le operazioni di lettura e scrittura sul database.

Il DAO riceve e restituisce sempre Entity — mai DTO.

È compito del **Service**, che vedremo a breve, fare da traduttore tra i DTO che arrivano dal Controller e le Entity che il DAO si aspetta.

☰ **In sintesi i tre tipi di classi hanno ruoli ben distinti:**

- Entity — rappresentano le tabelle del database
- DTO — rappresentano i dati scambiati con il client
- DAO — contengono la logica di accesso al database, parlano solo con le Entity

Le Conversioni — Da DTO a Entity e viceversa

Come abbiamo visto nell'architettura completa, il **Service** è il layer che fa da traduttore tra i due mondi:

- Riceve DTO dal Controller e li converte in Entity prima di passarli al DAO

- **Riceve Entity dal DAO e le converte in DTO prima di restituirle al Controller**

Questa responsabilità di conversione è del Service, **ma è buona pratica non scrivere la logica di conversione direttamente nei metodi del Service:**

- **diventa difficile da mantenere e non è riusabile.**

È preferibile invece creare una **classe di utility separata** con metodi statici di conversione:



Java

```
1
2
3 // metodi di trasformazione
4 public static Utente daDTOAEntity(UtenteDTO dto){
5     return new Utente(dto.getIdUtente(), dto.getNome(),
6         dto.getCognome(), dto.getMail(), dto.getTelefono());
7 }
8
9 public static UtenteDTO daEntityADto(Utente ut){
10     return new UtenteDTO(ut.getIdUtente(), ut.getNome(),
11         ut.getCognome(), ut.getMail())
12 }
```

Nota un dettaglio interessante nel secondo metodo:

- `Utente` (l'Entity) ha anche il campo `telefono`, ma `UtenteDTO` non lo include.
- Questo è esattamente uno dei motivi per cui DTO ed Entity sono classi separate:
 - **il DTO espone al client solo i dati necessari, nascondendo i campi che non devono uscire dal Model.**

👁 **La classe di utility viene tipicamente chiamata `Mapper` o `Converter`:**

un nome che riflette chiaramente il suo scopo.

I suoi metodi sono `static` perché non hanno bisogno di stato — sono pure funzioni di trasformazione.

Esempio Pratico — Gestione degli Utenti con Spring

Ora che abbiamo visto tutta l'architettura, mettiamo tutto insieme con un esempio pratico completo. L'esercizio simula un sistema di gestione degli utenti, implementando tutti i layer dell'architettura che abbiamo studiato: Entity, DTO, Mapper, DAO, Service e Controller.

Nota:

Come anticipato nella sezione sul Data Layer, al posto di un database reale utilizziamo una **HashMap** per simulare la persistenza — esattamente come in **Flask** usavamo i **dizionari Python**.

Le HashMap **sono composte da coppie chiave-valore che ricordano la struttura di una tabella SQL, dove la chiave corrisponde alla chiave primaria e il valore corrisponde alla riga.**

1. Entity — Utente



Java

```
1  public class Utente {
2      private int idUtente;
3      private String nome, cognome, mail, telefono;
4
5      public Utente() { }
6
7      public Utente(int idUtente, String nome, String cognome,
8          String mail, String telefono) {
9          this.idUtente = idUtente;
10         this.nome = nome;
11         this.cognome = cognome;
12         this.mail = mail;
13         this.telefono = telefono;
14     }
15     // getter e setter...
16 }
```

L'Entity Utente è la rappresentazione Java della tabella utenti nel database:

- **ogni campo corrisponde a una colonna e ogni oggetto istanziato corrisponde a una riga.**
- È il tipo di oggetto che vive esclusivamente nel Model:

- **il Controller non lo vedrà mai direttamente — vedrà solo i DTO.**

Ha due costruttori, ed entrambi hanno uno scopo preciso:

1. Il costruttore vuoto:

- **è obbligatorio in tutti gli oggetti che Spring deve gestire — sia Entity che DTO.**
- **Questo perché quando Spring riceve una richiesta HTTP con un payload JSON, deve deserializzare quel JSON in un oggetto Java.**
- **Per farlo usa il costruttore vuoto per istanziare prima l'oggetto, e poi popola i campi uno per uno tramite i setter.**
- **Senza il costruttore vuoto Spring non saprebbe come istanziare la classe e lancerebbe un errore.**

Questo avviene quando il client invia una richiesta HTTP con un payload JSON, ad esempio una `POST /registra` con i dati dell'utente nel body. In quel momento Spring deve trasformare quel JSON in un oggetto `UtenteDTO` :

- **per farlo istanzia prima l'oggetto tramite il costruttore vuoto, e poi popola i campi uno per uno tramite i setter.**

Non serve invece per le operazioni di lettura come `selectAll()` o `selectById()` — in quei casi Spring non deve creare nulla:

- **gli oggetti esistono già nella HashMap e vengono semplicemente restituiti.**

📌 Nota mentale:

- **Serializzazione:**
 - **da oggetto Java a JSON (in uscita, quando Spring trasforma il tuo `UtenteDTO` nella risposta JSON che manda al client)**
 - **Deserializzazione:**
 - **da JSON a oggetto Java (in ingresso, quando Spring trasforma il JSON della richiesta del client in un oggetto `UtenteDTO`)**
- Quindi quando il client fa una `POST /registra` inviando questo JSON:

```
{ }  JSON
```

```
1  {
```

```
2     "idUtente": 5,  
3     "nome": "Luca",  
4     "cognome": "Bianchi",  
5     "mail": "luca@gmail.com",  
6     "telefono": "333123456"  
7 }
```

Spring lo **deserializza**:

- **crea un oggetto** `UtenteDTO` **vuoto tramite il costruttore vuoto e poi popola i campi uno per uno usando i setter, abbinando i nomi delle chiavi JSON ai nomi dei campi Java.**

Viceversa quando il Controller restituisce un `UtenteDTO`, Spring lo **serializza** automaticamente grazie a `@RestController`:



Java

```
1 // il Controller restituisce questo oggetto Java...  
2 new UtenteDTO(5, "Luca", "Bianchi", "luca@gmail.com",  
   "333123456")
```



JSON

```
1 // ...e Spring lo converte automaticamente in questo JSON  
   nella response  
2 {  
3     "idUtente": 5,  
4     "nome": "Luca",  
5     "cognome": "Bianchi",  
6     "mail": "luca@gmail.com",  
7     "telefono": "333123456"  
8 }
```

☰ In sintesi:

il costruttore vuoto non è una scelta stilistica — è un **requisito tecnico** di Spring per la deserializzazione JSON → Java. Senza di esso Spring lancerebbe un errore al momento della richiesta.

1. Il costruttore completo:

- è quello che usiamo noi nel codice applicativo:
 - **ad esempio nel DAO quando popoliamo la HashMap con i dati di esempio, o nel Mapper quando costruiamo un nuovo oggetto a partire da un DTO.**

Nota mentale:

Il costruttore completo serve ogni volta che sei **tu** a creare esplicitamente un oggetto con tutti i campi già valorizzati. Nel flusso di `registra()` ad esempio:

1. **Spring riceve il JSON e usa il costruttore vuoto per creare l'UtenteDTO**
2. **Il Mapper usa il costruttore completo per creare l'Utente Entity a partire dal DTO:**



Java

```
1 return new Utente(dto.getIdUtente(), dto.getNome(),  
    dto.getCognome(), dto.getMail(), dto.getTelefono());
```

3. **Il DAO usa il costruttore completo per popolare la HashMap con i dati di esempio:**



Java

```
1 mappa.put(1, new Utente(1, "Marco", "PierLeoni",  
    "marcoP@gmail.com", "1321321321"));
```

in sintesi:

- **Costruttore vuoto → lo usa Spring per deserializzare il JSON**
- **Costruttore completo → lo usiamo noi quando creiamo oggetti esplicitamente nel codice**

2. DTO — `UtenteDTO` e `NomeCognomeDTO`



Java


```

1  public class UtenteDTO {
2      private int idUtente;
3      private String nome, cognome, mail, telefono;
4
5      public UtenteDTO() { }
6
7      public UtenteDTO(int idUtente, String nome, String cognome,
8          String mail, String telefono) {
9          // ...
10     }
11     // getter e setter...
12 }

```



Java

```

1  public class NomeCognomeDTO {
2      private String nome, cognome;
3
4      public NomeCognomeDTO() { }
5
6      public NomeCognomeDTO(String nome, String cognome) {
7          // ...
8      }
9      // getter e setter...
10 }

```

In questo esercizio abbiamo **due DTO**: ed è un esempio perfetto di perché i DTO esistono.

- Abbiamo detto che i DTO corrispondono alle aggregazione/query tra più tabelle o nella singola tabella, a differenza delle entity che corrispondono alle tabelle stesse del database.
- **Quindi** UtenteDTO **trasporta tutti i dati dell'utente verso il client,**
- **mentre** NomeCognomeDTO **trasporta solo nome e cognome.**

Perché non restituire semplicemente un UtenteDTO con i campi mail e telefono vuoti? **Per 3 motivi:**

1. Chiarezza:

- un DTO dedicato con solo nome e cognome è esplicito:
 - **il client sa esattamente cosa aspettarsi. È l'equivalente di una** `SELECT nome, cognome FROM utenti` **invece di una** `SELECT *`.

2. Sicurezza:

- se si restituisse un `UtenteDTO` con `mail` e `telefono` a `null`, il client non saprebbe distinguere se quei campi sono vuoti per scelta o per un errore lato server.
- Con un `NomeCognomeDTO` quei campi non esistono proprio nel JSON — nessuna ambiguità.

3. Coerenza della response:

- il JSON restituito al client conterrà solo i campi effettivamente significativi:

```
{ } JSON

1 // ✓ NomeCognomeDTO - pulito e esplicito
2 {
3     "nome": "Marco",
4     "cognome": "PierLeoni"
5 }
6
7 // ✗ UtenteDTO con campi vuoti - ambiguo e incoerente
8 {
9     "idUtente": 0,
10    "nome": "Marco",
11    "cognome": "PierLeoni",
12    "mail": null,
13    "telefono": null
14 }
```

Anche i DTO hanno il costruttore vuoto:

- Spring ne ha bisogno per le stesse ragioni dell'Entity: deserializzare il JSON in ingresso in un oggetto Java.

4. Mapper — Mapper



Java

```
1 public class Mapper {
2
3     // da DTO a Entity - usato dal Service prima di chiamare il
    DAO
4     public static Utente daUtenteDTOAUtente(UtenteDTO dto) {
5         return new Utente(
6             dto.getIdUtente(), dto.getNome(), dto.getCognome(),
```

```

7         dto.getEmail(), dto.getTelefono()
8     );
9 }
10
11 // da Entity a DTO – usato dal Service dopo aver ricevuto
    dati dal DAO
12     public static UtenteDTO daUtenteAUtenteDTO(Utente utente) {
13         return new UtenteDTO(
14             utente.getIdUtente(), utente.getNome(),
15             utente.getCognome(),
16             utente.getEmail(), utente.getTelefono()
17         );
18     }
19 }

```

Il Mapper è la classe di utility:

- **contiene le funzioni di conversione tra Entity e DTO.**
I metodi sono `static` perché non hanno bisogno di stato:
- **sono pure funzioni di trasformazione.**
- **È buona pratica centralizzare queste conversioni in una classe dedicata invece di spargerle nel codice del Service.**

I due metodi coprono le due direzioni del flusso che abbiamo visto nell'architettura:

1. `daUtenteDTOAUtente()` :
 - **viene invocato dal Service prima di chiamare il DAO.**
 - **Il Service riceve un `UtenteDTO` dal Controller, ma il DAO lavora solo con Entity — quindi il Mapper costruisce un nuovo oggetto `Utente` estraendo i valori dal DTO tramite i getter e passandoli al costruttore completo dell'Entity.**
2. `daUtenteAUtenteDTO()` :
 - **viene invocato dal Service dopo aver ricevuto i dati dal DAO.**
 - **Il DAO restituisce un'Entity `Utente`, ma il Controller deve ricevere un DTO — quindi il Mapper fa il percorso inverso, costruendo un `UtenteDTO` a partire dall'Entity.**
3. DAO — `DAOUtenteMappa`



```

1  public class DAOUtenteMappa {
2      private Map<Integer, Utente> mappa = new HashMap<>();
3
4      public DAOUtenteMappa() {
5          mappa.put(1, new Utente(1, "Marco", "PierLeoni",
6      "marcoP@gmail.com", "1321321321"));
7          mappa.put(2, new Utente(2, "Pier", "Damien",
8      "pierD@gmail.com", "12332131"));
9          mappa.put(3, new Utente(3, "Ale", "Cole",
10     "aleC@gmail.com", "11233121"));
11         mappa.put(4, new Utente(4, "Barbara", "Cole",
12     "barbaraC@gmail.com", "11233121"));
13     }
14
15     public boolean insert(Utente utente) {
16         if(mappa.containsKey(utente.getIdUtente())) return false;
17         mappa.put(utente.getIdUtente(), utente);
18         return true;
19     }
20
21     public List<Utente> selectAll() {
22         return new ArrayList<>(mappa.values());
23     }
24
25     public Utente selectById(Integer idUtente) {
26         return mappa.get(idUtente);
27     }
28
29     public Utente delete(Integer idUtente) {
30         return mappa.remove(idUtente);
31     }
32 }

```

Il DAO simula la persistenza tramite una `HashMap<Integer, Utente>` dove la chiave è l' `id` dell'utente — esattamente come una chiave primaria in SQL.

 Il costruttore popola la mappa con quattro utenti di esempio, simulando i dati già presenti nel database.

Questa è una scelta pratica e non architetturale:

- popolare la mappa nel costruttore significa che ogni volta che Spring avvia il server, la mappa nasce già popolata — senza

bisogno di reinserire manualmente i dati tramite POST su [Postman](#) ad ogni riavvio.

In un'applicazione reale con un database vero questo problema non si pone, perché i dati persistono indipendentemente dal ciclo di vita del server.

I metodi rispecchiano le operazioni CRUD che abbiamo già visto con JDBC:

- `insert()` — **verifica che la chiave non esista già prima di inserire, restituisce** `false` **in caso di duplicato**
- `selectAll()` — **restituisce tutti i valori della mappa come** `ArrayList`
- `selectById()` — **estrae il valore associato alla chiave tramite** `mappa.get()`
- `delete()` — **rimuove la coppia chiave-valore tramite** `mappa.remove()`, **restituendo l'oggetto rimosso**

Nota: il DAO lavora **esclusivamente con Entity** — non sa nulla dei DTO. È compito del Service fare la conversione.

5. Service — `UtenteService`



Java

```
1  @Service
2  public class UtenteService {
3      private DAOUtenteMappa dao = new DAOUtenteMappa();
4
5      public boolean registra(UtenteDTO dto) {
6          Utente utente = Mapper.daUtenteDTOAUtente(dto);
7          return dao.insert(utente);
8      }
9
10     public UtenteDTO cercaPerId(int id) {
11         Utente utente = dao.selectById(id);
12         if(utente != null) return
Mapper.daUtenteAUtenteDTO(utente);
13         else return null;
14     }
15
16     public List<UtenteDTO> selectAll() {
17         List<Utente> listaUtenti = dao.selectAll();
```

```

18         return listaUtenti.stream()
19             .map(t -> Mapper.daUtenteAUtenteDTO(t))
20             .collect(Collectors.toList());
21     }
22
23     public UtenteDTO deleteById(int id) {
24         Utente ut = dao.delete(id);
25         if(ut == null) return null;
26         return Mapper.daUtenteAUtenteDTO(ut);
27     }
28
29     public UtenteDTO updateEmail(int id, String email) {
30         Utente ut = dao.selectById(id);
31         if(ut == null) return null;
32         ut.setMail(email);
33         return Mapper.daUtenteAUtenteDTO(ut);
34     }
35
36     public List<NomeCognomeDTO> getNomiCognomi() {
37         return dao.selectAll().stream()
38             .map(t -> new NomeCognomeDTO(t.getNome(),
39                 t.getCognome()))
40             .collect(Collectors.toList());
41     }

```

Il Service è il layer più ricco di questo esercizio:

- **contiene tutta la logica applicativa e fa da traduttore tra Controller e DAO.**

Vale la pena analizzare alcuni metodi nel dettaglio:

`registra()` — **riceve un UtenteDTO dal Controller, lo converte in Entity tramite il Mapper, e lo passa al DAO per l'inserimento. Il pattern è sempre lo stesso: DTO in entrata, Entity verso il DAO.**

`cercaPerId()` — **chiama il DAO, riceve un Entity, e lo converte in DTO prima di restituirlo al Controller. Gestisce anche il caso in cui l'utente non esista restituendo `null`.**

`selectAll()` — **usa le Stream API per convertire la lista di Entity in lista di DTO in modo compatto. `stream().map().collect()` è il pattern Java**

moderno per trasformare collezioni — equivale a un for-each ma più leggibile.

`updateEmail()` :

- recupera l'Entity dal DAO tramite `selectById()` , aggiorna il campo `mail` tramite setter, e restituisce il DTO aggiornato.

② Vale la pena soffermarsi su un dettaglio importante: non viene chiamato nessun metodo di "update" sul DAO, eppure la HashMap si aggiorna automaticamente. Perché?

Quando `dao.selectById(id)` restituisce un oggetto `Utente` , non restituisce una copia:

- restituisce il riferimento allo stesso oggetto che vive dentro la HashMap.**

Quindi quando si chiama `ut.setMail(email)` su quel riferimento, si sta modificando direttamente l'oggetto originale dentro la mappa — esattamente come il **pass by reference** che abbiamo visto in Java base:

- passare un oggetto significa passare il riferimento all'originale, non una copia.**
- Qualsiasi modifica fatta tramite quel riferimento si riflette automaticamente sull'oggetto originale, e di conseguenza sul valore nella HashMap.**

`getNomiCognomi()` — è un esempio perfetto dell'utilità dei DTO dedicati:

- invece di restituire oggetti `UtenteDTO` con i campi `mail` e `telefono` vuoti, costruisce direttamente oggetti `NomeCognomeDTO` con solo i campi necessari.**

6. Controller — `UtenteController`



Java

```
1 @RestController
2 @RequestMapping(path="/utenti")
3 public class UtenteController {
4
5     private UtenteService service = new UtenteService();
```

```

6
7     @PostMapping(path="/registra", consumes="application/json")
8     public boolean registra(@RequestBody UtenteDTO utente) {
9         return service.registra(utente);
10    }
11
12    @GetMapping(path="/elenco", produces="application/json")
13    public List<UtenteDTO> elencoUtenti() {
14        return service.selectAll();
15    }
16
17    @DeleteMapping(path="/elimina/{id}",
18    produces="application/json")
19    public UtenteDTO delete(@PathVariable int id) {
20        return service.deleteById(id);
21    }
22
23    @PatchMapping(path="/aggiorna/{idUtente}",
24    produces="application/json")
25    public UtenteDTO updateE(@PathVariable int idUtente, String
26    email) {
27        return service.updateEmail(idUtente, email);
28    }
29
30    @GetMapping(path="/cerca/{idUtente}",
31    produces="application/json")
32    public UtenteDTO cerca(@PathVariable int idUtente) {
33        return service.cercaPerId(idUtente);
34    }
35
36    @GetMapping(path="/nomiCognomi", produces="application/json")
37    public List<NomeCognomeDTO> getNomiCognomi() {
38        return service.getNomiCognomi();
39    }
40
41    }

```

Il Controller è il layer più snello — ogni metodo fa una sola cosa: ricevere la richiesta e delegare al Service. Non c'è logica, non ci sono conversioni, non ci sono accessi al DAO. È il centralino puro che abbiamo descritto nell'architettura.

Vale la pena notare come ogni metodo usa il verbo HTTP corretto per l'operazione che esegue:

- `@PostMapping` per `registra()` — inserimento

- `@GetMapping` per `elencoUtenti()`, `cerca()`, `getNomiCognomi()` — lettura
- `@DeleteMapping` per `delete()` — cancellazione
- `@PatchMapping` per `updateE()` :
 - **Si usa `@PatchMapping` e non `@PutMapping` perché stiamo aggiornando solo la mail — non l'intero oggetto `Utente`.**
 - **Usare `@PutMapping` implicherebbe che il client stia inviando un oggetto completo per sostituire quello esistente.**
 - **`@PatchMapping` è quindi la scelta semanticamente corretta per modifiche parziali.**

PUT **vs** PATCH :

`PUT` sostituisce l'intera risorsa, `PATCH` la aggiorna parzialmente.
Entrambi sono generalmente idempotenti:

- **mandare la stessa richiesta più volte produce sempre lo stesso risultato — anche se per `PATCH` questa garanzia dipende dall'implementazione.**

I parametri vengono passati in tre modi diversi, a seconda del metodo:

- `@RequestBody` — **i dati arrivano nel body della richiesta in JSON (es. `registra`)**
- `@PathVariable` — **i dati arrivano direttamente nell'URL (es. `/cerca/3`)**
- **Query parameter** — **i dati arrivano accodati all'URL con `?` (es. `updateE` per l'email)**